

Programação Orientada a Objetos em Java: Hierarquia e Relacionamento de Classes

Prof. Dani Juliano Czelusniak

Atualizado em 2026

1 Introdução aos Relacionamentos Orientados a Objetos

Na Programação Orientada a Objetos (POO), sistemas complexos raramente são construídos com classes isoladas. A verdadeira força do paradigma reside em como as instâncias (objetos) interagem e se conectam para simular processos do mundo real.

A hierarquia e o relacionamento entre classes permitem reaproveitamento de código, organização lógica e modelagem precisa das regras de negócio. Os quatro principais tipos de relacionamentos estruturais que estudaremos são: Associação Simples, Herança, Agregação e Composição.

2 Associação Simples

A associação simples é o relacionamento mais básico. Ocorre quando duas classes independentes se conectam estruturalmente, indicando que uma "conhece" a outra ou precisa da outra para executar alguma ação. Nenhuma das classes é "dona" da outra; seus ciclos de vida são independentes.

Na notação UML, é representada por uma linha contínua conectando as duas classes.

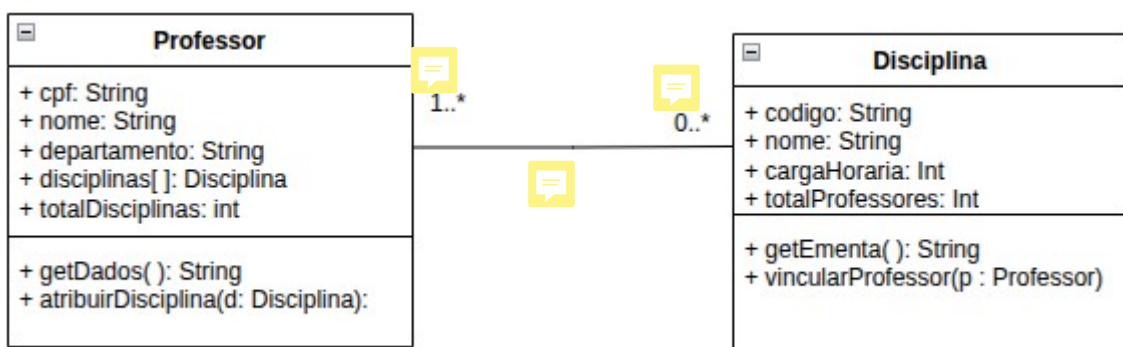


Figura 1: Diagrama de Associação Simples: Professor e Disciplina.

```
1 class Disciplina {
2     public String codigo;
```

```

3 public String nome;
4 public int cargaHoraria;
5 public Professor[] professores;
6 public int totalProfessores;
7
8 public Disciplina(String codigo, String nome, int cargaHoraria) {
9     this.codigo = codigo;
10    this.nome = nome;
11    this.cargaHoraria = cargaHoraria;
12    this.professores = new Professor[5];
13    this.totalProfessores = 0;
14 }
15
16 public String getEmenta() {
17     return "Ementa: " + this.nome + " (" + this.cargaHoraria + "h
18 );";
19 }
20
21 public void vincularProfessor(Professor p) {
22     if (this.totalProfessores < this.professores.length) {
23         this.professores[this.totalProfessores] = p;
24         this.totalProfessores++;
25     }
26 }
27
28 class Professor {
29     public String cpf;
30     public String nome;
31     public String departamento;
32     public Disciplina[] disciplinas;
33     public int totalDisciplinas;
34
35     public Professor(String cpf, String nome, String departamento) {
36         this.cpf = cpf;
37         this.nome = nome;
38         this.departamento = departamento;
39         this.disciplinas = new Disciplina[5];
40         this.totalDisciplinas = 0;
41     }
42
43     public String getDados() {
44         return "Prof. " + this.nome + " (Depto: " + this.departamento
45 + ")";
46     }
47
48     public void atribuirDisciplina(Disciplina d) {
49         if (this.totalDisciplinas < this.disciplinas.length) {
50             this.disciplinas[this.totalDisciplinas] = d;
51             this.totalDisciplinas++;
52         }
53     }
54 }

```

```

52     }
53 }
54
55 // Classe Principal para execucao
56 public class Main {
57     public static void main(String[] args) {
58         Professor prof1 = new Professor("111", "Dani Juliano", "
Computacao");
59         Disciplina disc1 = new Disciplina("CS101", "POO em Java", 60)
;
60
61         // Estabelecendo a associacao entre os dois objetos
independentes
62         prof1.atribuirDisciplina(disc1);
63         disc1.vincularProfessor(prof1);
64
65         System.out.println("--- Teste de Associacao ---");
66         System.out.println(prof1.getDados());
67         System.out.println(disc1.getEmenta());
68         System.out.println("A disciplina " + disc1.nome + " possui "
+
69                             disc1.totalProfessores + " professor(es)
vinculado(s).");
70     }
71 }

```

Listing 1: Exemplo de Associação Simples (Executável)

3 Herança (Generalização/Especialização)

A Herança permite criar uma nova classe (subclasse ou classe filha) a partir de uma classe existente (superclasse ou classe mãe). É um relacionamento do tipo "É-UM" (*is-a*). A subclasse herda todos os atributos e métodos acessíveis da superclasse, podendo adicionar novos ou reescrever (sobrescrever) os existentes.

Na UML, é representada por uma seta com a ponta vazada apontando para a superclasse.

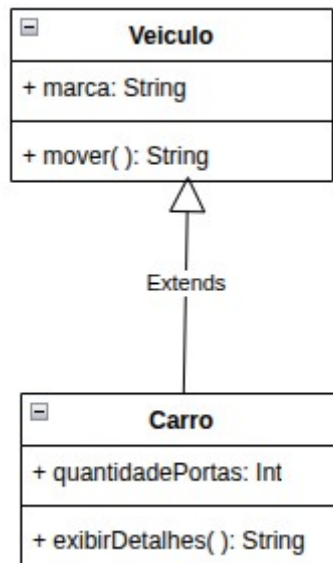


Figura 2: Diagrama de Herança: Veiculo e Carro.

```
1 class Veiculo {
2     protected String marca;
3
4     public Veiculo(String marca) {
5         this.marca = marca;
6     }
7
8     public void mover() {
9         System.out.println("O veiculo da marca " + this.marca + "
10        esta em movimento.");
11    }
12 }
13 // Carro E-UM Veiculo
14 class Carro extends Veiculo {
15     private int quantidadePortas;
16
17     public Carro(String marca, int portas) {
18         super(marca); // Chama o construtor da superclasse
19         this.quantidadePortas = portas;
20     }
21
22     public void exibirDetalhes() {
23         System.out.println("Carro " + this.marca + " com " +
24                             this.quantidadePortas + " portas.");
25     }
26 }
27
28 // Classe Principal para execucao
29 public class Main {
30     public static void main(String[] args) {
```

```

31     System.out.println("--- Teste de Heranca ---");
32
33     Carro meuCarro = new Carro("Toyota", 4);
34
35     meuCarro.exibirDetalhes(); // Chamando metodo especifico da
36     subclasse
37     meuCarro.mover();          // Chamando metodo herdado da
38     superclasse
39 }

```

Listing 2: Exemplo de Herança (Executável)

4 Agregação

A Agregação é um relacionamento do tipo "TEM-UM" (*has-a*) ou "PARTE-TODO". É uma forma específica de associação onde uma classe (Todo) contém objetos de outra classe (Parte). No entanto, a "Parte" pode existir de forma independente do "Todo". Se o objeto "Todo" for destruído, os objetos "Parte" continuam existindo.

Na UML, é representada por uma linha com um losango vazado (branco) no lado da classe que representa o "Todo".

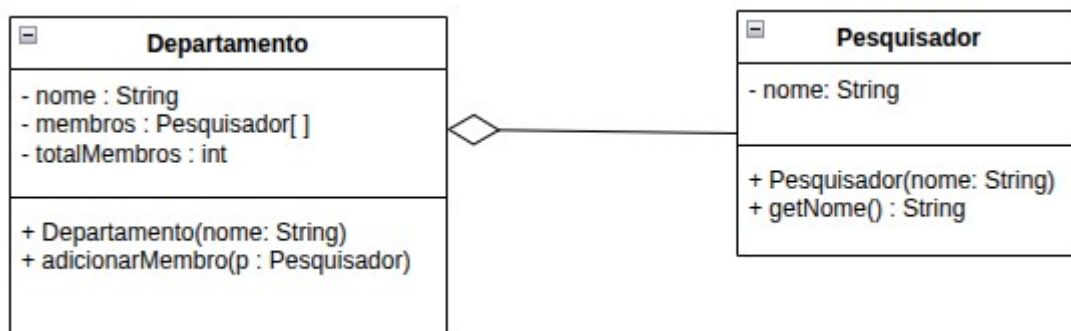


Figura 3: Diagrama de Agregação: Departamento tem Professores.

```

1 class Pesquisador {
2     private String nome;
3
4     public Pesquisador(String nome) {
5         this.nome = nome;
6     }
7
8     public String getNome() {
9         return this.nome;
10    }
11 }
12
13 class Departamento {
14     private String nome;
15     private Pesquisador[] membros;

```

```

16     private int totalMembros;
17
18     public Departamento(String nome) {
19         this.nome = nome;
20         this.membros = new Pesquisador[20];
21         this.totalMembros = 0;
22     }
23
24     public void adicionarMembro(Pesquisador p) {
25         if (this.totalMembros < this.membros.length) {
26             this.membros[this.totalMembros] = p;
27             this.totalMembros++;
28             System.out.println(p.getNome() + " adicionado ao depto "
+ this.nome);
29         } else {
30             System.out.println("Departamento cheio!");
31         }
32     }
33
34     public void listarMembros() {
35         System.out.println("Membros do depto " + this.nome + ":");
36         for (int i = 0; i < this.totalMembros; i++) {
37             System.out.println("- " + this.membros[i].getNome());
38         }
39     }
40 }
41
42 // Classe Principal para execucao
43 public class Main {
44     public static void main(String[] args) {
45         System.out.println("--- Teste de Agregacao ---");
46
47         // As partes sao instanciadas de forma totalmente
independente do Todo
48         Pesquisador p1 = new Pesquisador("Alan Turing");
49         Pesquisador p2 = new Pesquisador("Ada Lovelace");
50
51         Departamento depto = new Departamento("Ciencia da Computacao"
);
52
53         // Agregando as partes ao objeto principal
54         depto.adicionarMembro(p1);
55         depto.adicionarMembro(p2);
56
57         depto.listarMembros();
58     }
59 }

```

Listing 3: Exemplo de Agregação (Executável)

5 Composição

A Composição é uma forma mais forte de agregação. É um relacionamento "PARTE-TODO" rigoroso, caracterizado por dependência de ciclo de vida. A "Parte" não faz sentido isoladamente e não sobrevive se o "Todo" for destruído. A classe "Todo" é responsável por instanciar e destruir a classe "Parte".

Na UML, é representada por uma linha com um losango preenchido (preto) no lado do "Todo".

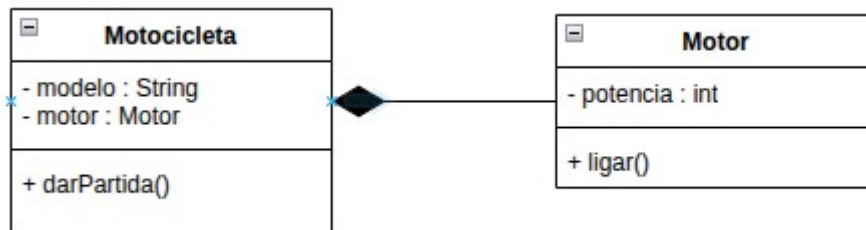


Figura 4: Diagrama de Composição: Pedido e ItemPedido.

```
1 class Motor {
2     private int potencia;
3
4     public Motor(int potencia) {
5         this.potencia = potencia;
6     }
7
8     public void ligar() {
9         System.out.println("Motor de " + this.potencia + "
10        cilindradas ligado!");
11    }
12 }
13
14 class Motocicleta {
15     private String modelo;
16     private Motor motor;
17
18     public Motocicleta(String modelo, int potenciaMotor) {
19         this.modelo = modelo;
20         // Na composicao, o Todo cria a Parte ativamente no seu
21         interior
22         this.motor = new Motor(potenciaMotor);
23     }
24
25     public void darPartida() {
26         System.out.println("Dando partida na " + this.modelo + "...")
27         ;
28         this.motor.ligar();
29     }
30 }
31
32 // Classe Principal para execucao
```

```
30 public class Main {
31     public static void main(String[] args) {
32         System.out.println("--- Teste de Composicao ---");
33
34         // O main nao cria o motor. Ele apenas instancia a moto.
35         // O ciclo de vida do motor esta totalmente submisso a moto.
36         Motocicleta moto = new Motocicleta("Custom 1200cc", 1200);
37
38         moto.darPartida();
39     }
40 }
```

Listing 4: Exemplo de Composição (Executável)

6 Lista de Exercícios Resolvidos

Abaixo, encontram-se 8 exercícios focados nos quatro tipos de relacionamentos apresentados, formatados com suas respectivas classes Main para execução e testes práticos em compiladores online (como GDB Online ou JDoodle).

6.1 Nível Básico

Exercício 1 (Associação Simples): Crie as classes Pessoa e Endereco. Uma pessoa deve possuir um endereço e exibir seus dados completos.

```
1 class Endereco {
2     private String rua;
3     public Endereco(String rua) { this.rua = rua; }
4     public String getRua() { return this.rua; }
5 }
6
7 class Pessoa {
8     private String nome;
9     private Endereco endereco; // Associacao
10
11     public Pessoa(String nome, Endereco endereco) {
12         this.nome = nome;
13         this.endereco = endereco;
14     }
15
16     public void imprimir() {
17         System.out.println(nome + " mora em: " + endereco.getRua());
18     }
19 }
20
21 public class Main {
22     public static void main(String[] args) {
23         // Objetos criados de forma independente
24         Endereco end = new Endereco("Av. Sete de Setembro, 123");
25         Pessoa p = new Pessoa("Carlos", end); // Associando
26
27         p.imprimir();
28     }
29 }
```

Listing 5: Resolução Exercício 1

Exercício 2 (Herança): Implemente uma classe Animal com o método emitirSom(). Crie uma classe Cachorro que herda de Animal e sobrescreva o método.

```
1 class Animal {
2     public void emitirSom() {
3         System.out.println("Som genérico de animal");
4     }
5 }
6
7 class Cachorro extends Animal {
```

```

8      @Override
9      public void emitirSom() {
10         System.out.println("Au Au!");
11     }
12 }
13
14 public class Main {
15     public static void main(String[] args) {
16         Animal a = new Animal();
17         Cachorro c = new Cachorro();
18
19         System.out.print("Animal faz: ");
20         a.emitirSom();
21
22         System.out.print("Cachorro faz: ");
23         c.emitirSom();
24     }
25 }

```

Listing 6: Resolução Exercício 2

6.2 Nível Intermediário

Exercício 3 (Agregação): Crie uma classe Equipe e uma classe Jogador. A equipe deve possuir um vetor de jogadores, que são instanciados independentemente.

```

1 class Jogador {
2     private String nome;
3     public Jogador(String nome) { this.nome = nome; }
4     public String getNome() { return this.nome; }
5 }
6
7 class Equipe {
8     private String nome;
9     private Jogador[] jogadores;
10    private int totalJogadores;
11
12    public Equipe(String nome) {
13        this.nome = nome;
14        this.jogadores = new Jogador[22]; // Vetor com limite fixo
15        this.totalJogadores = 0;
16    }
17
18    // Jogador existe sem a Equipe
19    public void addJogador(Jogador j) {
20        if (this.totalJogadores < this.jogadores.length) {
21            this.jogadores[this.totalJogadores] = j;
22            this.totalJogadores++;
23        }
24    }
25
26    public void listar() {

```

```

27     System.out.println("Equipe " + nome + ":");
28     for(int i = 0; i < this.totalJogadores; i++) {
29         System.out.println("- " + this.jogadores[i].getNome());
30     }
31 }
32 }
33
34 public class Main {
35     public static void main(String[] args) {
36         Equipe eq = new Equipe("Santos FC");
37
38         // Criando partes independentes do Todo
39         Jogador j1 = new Jogador("Pelé");
40         Jogador j2 = new Jogador("Neymar");
41
42         // Agregando
43         eq.addJogador(j1);
44         eq.addJogador(j2);
45
46         eq.listar();
47     }
48 }

```

Listing 7: Resolução Exercício 3

Exercício 4 (Composição): Implemente Casa e Comodo. A casa cria seus cômodos internamente. Se a casa for deletada da memória, os cômodos não podem ser aproveitados.

```

1 class Comodo {
2     private String tipo;
3     public Comodo(String tipo) { this.tipo = tipo; }
4     public String getTipo() { return this.tipo; }
5 }
6
7 class Casa {
8     private Comodo[] comodos;
9
10    public Casa() {
11        this.comodos = new Comodo[2];
12        // A casa eh totalmente responsavel por instanciar a parte (
13        Composicao)
14        this.comodos[0] = new Comodo("Sala");
15        this.comodos[1] = new Comodo("Cozinha");
16    }
17
18    public void descrever() {
19        System.out.println("Casa composta pelos comodos:");
20        for (int i = 0; i < this.comodos.length; i++) {
21            System.out.println("- " + this.comodos[i].getTipo());
22        }
23    }
24 }

```

```

25 public class Main {
26     public static void main(String[] args) {
27         // O main apenas cria a Casa. Os comodos nascem submissos a
        ela.
28         Casa minhaCasa = new Casa();
29         minhaCasa.descrever();
30     }
31 }

```

Listing 8: Resolução Exercício 4

Exercício 5 (Herança e Polimorfismo): Implemente **Empregado** (salário base) e **Gerente** (salário base + bônus). Demonstre a chamada do método `getSalario()`.

```

1 class Empregado {
2     protected double salario;
3     public Empregado(double salario) { this.salario = salario; }
4     public double getSalario() { return this.salario; }
5 }
6
7 class Gerente extends Empregado {
8     private double bonus;
9     public Gerente(double salario, double bonus) {
10         super(salario);
11         this.bonus = bonus;
12     }
13     @Override
14     public double getSalario() {
15         return this.salario + this.bonus; // Usa dado herdado
16     }
17 }
18
19 public class Main {
20     public static void main(String[] args) {
21         Empregado emp = new Empregado(2500.0);
22         Gerente ger = new Gerente(4000.0, 1500.0);
23
24         System.out.println("Salario do Empregado: R$ " + emp.
        getSalario());
25         System.out.println("Salario do Gerente: R$ " + ger.getSalario
        ());
26     }
27 }

```

Listing 9: Resolução Exercício 5

Exercício 6 (Associação Múltipla): Modele **Medico** e **Paciente**. Um médico atende um paciente, e passa a ele uma prescrição (String).

```

1 class Paciente {
2     private String nome;
3     private String prescricao;
4     public Paciente(String nome) { this.nome = nome; }
5     public void setPrescricao(String p) { this.prescricao = p; }

```

```

6      public String getResumo() {
7          return "Paciente: " + this.nome + " | Receita: " + this.
prescricao;
8      }
9  }
10
11  class Medico {
12      private String nome;
13      public Medico(String nome) { this.nome = nome; }
14
15      public void atender(Paciente p, String remedio) {
16          System.out.println("Dr(a). " + this.nome + " esta atendendo o
paciente.");
17          p.setPrescricao(remedio); // Interacao direta
18      }
19  }
20
21  public class Main {
22      public static void main(String[] args) {
23          Medico medico = new Medico("House");
24          Paciente paciente = new Paciente("João");
25
26          medico.atender(paciente, "Dipirona 500mg");
27          System.out.println(paciente.getResumo());
28      }
29  }

```

Listing 10: Resolução Exercício 6

6.3 Nível Avançado

Exercício 7 (Composição e Agregação Juntas): Crie uma Universidade. Ela **compõe** Departamentos (criados no construtor da universidade) e **agrega** Professores (recebidos via parâmetro).

```

1  class Professor {
2      private String nome;
3      public Professor(String nome) { this.nome = nome; }
4      public String getNome() { return this.nome; }
5  }
6
7  class Departamento {
8      private String area;
9      public Departamento(String area) { this.area = area; }
10     public String getArea() { return this.area; }
11 }
12
13 class Universidade {
14     private String nome;
15     private Departamento deptExatas; // Composicao
16     private Professor[] corpoDocente; // Agregacao
17     private int totalProfessores;

```

```

18
19 public Universidade(String nome) {
20     this.nome = nome;
21     // Ciclo de vida dependente (Composicao):
22     this.deptExatas = new Departamento("Computacao");
23
24     this.corpoDocente = new Professor[50];
25     this.totalProfessores = 0;
26 }
27
28 // Ciclo de vida independente (Agregacao):
29 public void contratar(Professor p) {
30     if(this.totalProfessores < this.corpoDocente.length) {
31         this.corpoDocente[this.totalProfessores] = p;
32         this.totalProfessores++;
33     }
34 }
35
36 public void exibirInfo() {
37     System.out.println("Universidade: " + this.nome);
38     System.out.println("Departamento Interno: " + this.deptExatas
39 .getArea());
40     System.out.println("Professores contratados:");
41     for(int i = 0; i < this.totalProfessores; i++) {
42         System.out.println("- " + this.corpoDocente[i].getNome())
43 ;
44     }
45 }
46
47 public class Main {
48     public static void main(String[] args) {
49         Universidade uni = new Universidade("Tech University");
50
51         Professor p1 = new Professor("Alan Turing");
52         Professor p2 = new Professor("Ada Lovelace");
53
54         uni.contratar(p1);
55         uni.contratar(p2);
56
57         uni.exibirInfo();
58     }
59 }

```

Listing 11: Resolução Exercício 7

Exercício 8 (Ecossistema E-commerce Completo): Implemente uma estrutura onde um Pedido compõe múltiplos objetos ItemPedido. Cada ItemPedido tem uma associação simples com um Produto. O Pedido pertence a um Cliente (Agregação).

```

1 class Cliente {
2     private String nome;
3     public Cliente(String n) { this.nome = n; }

```

```

4     public String getNome() { return this.nome; }
5 }
6
7 class Produto {
8     private String descricao;
9     private double preco;
10    public Produto(String d, double p) { this.descricao = d; this.
11    preco = p; }
12    public double getPreco() { return preco; }
13    public String getDescricao() { return descricao; }
14 }
15
16 class ItemPedido {
17     private Produto prod; // Associacao Simples
18     private int qtd;
19
20     public ItemPedido(Produto p, int q) { this.prod = p; this.qtd = q
21     ; }
22     public double getTotal() { return prod.getPreco() * qtd; }
23     public String getResumo() {
24         return qtd + "x " + prod.getDescricao() + " (R$ " + getTotal
25         () + ")";
26     }
27 }
28
29 class Pedido {
30     private Cliente dono; // Agregacao
31     private ItemPedido[] itens; // Composicao
32     private int totalItens;
33
34     public Pedido(Cliente c) {
35         this.dono = c;
36         this.itens = new ItemPedido[10]; // Vetor nativo
37         this.totalItens = 0;
38     }
39
40     // O pedido cria os itens ativamente (Composicao)
41     public void addItem(Produto p, int qtd) {
42         if (this.totalItens < this.itens.length) {
43             this.itens[this.totalItens] = new ItemPedido(p, qtd);
44             this.totalItens++;
45         }
46     }
47
48     public void imprimirCupom() {
49         System.out.println("=== Cupom Fiscal ===");
50         System.out.println("Cliente: " + this.dono.getNome());
51         double total = 0;
52         for(int i = 0; i < this.totalItens; i++) {
53             System.out.println("- " + this.itens[i].getResumo());
54             total += this.itens[i].getTotal();
55         }
56     }
57 }

```

```

52     }
53     System.out.println("Total a pagar: R$ " + total);
54 }
55 }
56
57 public class Main {
58     public static void main(String[] args) {
59         Cliente cli = new Cliente("Ana");
60
61         Produto p1 = new Produto("Notebook", 3500.0);
62         Produto p2 = new Produto("Mouse Wireless", 150.0);
63
64         Pedido ped = new Pedido(cli);
65
66         ped.addItem(p1, 1);
67         ped.addItem(p2, 2);
68
69         ped.imprimirCupom();
70     }
71 }

```

Listing 12: Resolução Exercício 8